

PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NoSQL / MongoDB

PRACTICAL FILE

PRACTICAL NO. 2

WORKING WITH MONGODB DOCUMENTS AND COLLECTIONS

SUBMITTED TO – _MR HARENDRA SINGH_ RAJPOOT

ROLL NO -1250258270 (28)

SUBMITTED BY – Mohd Ashar Abbas

BCA – Data Science & Artificial Intelligence

SEMESTER – ODD (3rd)

UNIVERSITY / COLLEGE – BBD UNIVERSITY

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my mentor and faculty members for providing me the opportunity to perform this MongoDB practical. This experiment helped me understand document insertion, retrieval, modification and deletion using MongoDB CRUD operations. The practical was

performed on a Windows system using MongoDB Community Server, MongoDB Shell (mongosh), and MongoDB Compass.

PRACTICAL NO. 2

Working with MongoDB Documents and Collections

OBJECTIVE

To work with MongoDB documents and collections by performing insert, view, update and delete operations using CRUD commands.

REQUIREMENTS

- MongoDB Community Edition / MongoDB server
- MongoDB Shell (mongosh)
- MongoDB Compass (optional)
- StudentDB database created in Practical 1

THEORY

CRUD stands for Create, Read, Update and Delete. MongoDB provides methods such as insertOne(), insertMany(), find(), updateOne(), updateMany(), deleteOne() and deleteMany() to manage documents.

PROCEDURE / COMMANDS USED

1. Select Database

```
use StudentDB
```

2. Insert a Single Document

```
db.students.insertOne({  
  rollNo: 101, name:  
  "Aarav", course: "BCA",  
  semester: 3, marks: 82,  
  attendance: 92 })
```

3. Insert Multiple Documents

```
db.students.insertMany([ {rollNo: 102, name: "Aditi", course: "BCA", semester: 3,  
marks: 91, attendance: 95},  
{rollNo: 103, name: "Aditya", course: "BCA", semester: 3, marks: 76, attendance: 81},  
{rollNo: 104, name: "Akash", course: "BCA", semester: 3, marks: 88, attendance: 88}  
// additional student records were inserted during execution  
)
```

4. View Documents

```
db.students.find()  
db.students.find({course: "BCA"})  
db.students.find({marks: {$gt: 80}})
```

5. Update a Document

```
db.students.updateOne(  
  {rollNo: 101},  
  {$set: {marks: 90}}  
)  
db.students.findOne({rollNo: 101})
```

6. Delete a Document

```
db.students.deleteOne({rollNo: 140})  
db.students.findOne({rollNo: 140})
```

7. Verify Collection

```
show collections  
db.students.countDocuments()
```

SCREENSHOT / EXPERIMENTAL EVIDENCE

```
-----
The server generated these startup warnings when booting
2026-09-19T18:02:44.917+05:30: Access control is not enabled for the database. Re
-----

test> use StudentDB
switched to db StudentDB
StudentDB> db.createCollection("students")
{ ok: 1 }
StudentDB> db.createCollection("students")
{ ok: 1 }
StudentDB> show collections
students
StudentDB> db.students.insertOne({
  |   rollNo: 101,
  |   name: "Aarav",
  |   course: "BCA",
  |   semester: 3,
  |   marks: 82,
  |   attendance: 92
  | })
{
  acknowledged: true,
  insertedId: ObjectId('6aae8c72d26a80948c479298')
}
StudentDB>
```

Figure 1: Single student document inserted successfully using insertOne(). The output shows acknowledged: true and an inserted ObjectId.

SCREENSHOT / EXPERIMENTAL EVIDENCE

```
StudentDB> db.students.insertMany([
  {rollNo:102,name:"Aditi",course:"BCA",semester:3,marks:91,attendance:95},
  {rollNo:103,name:"Aditya",course:"BCA",semester:3,marks:76,attendance:81},
  {rollNo:104,name:"Akash",course:"BCA",semester:3,marks:88,attendance:88},
  {rollNo:105,name:"Ananya",course:"BCA",semester:3,marks:69,attendance:76},
  {rollNo:106,name:"Ankit",course:"BCA",semester:3,marks:95,attendance:97},
  {rollNo:107,name:"Arjun",course:"BCA",semester:3,marks:73,attendance:79},
  {rollNo:108,name:"Ayush",course:"BCA",semester:3,marks:84,attendance:91},
  {rollNo:109,name:"Bhavna",course:"BCA",semester:3,marks:79,attendance:84},
  {rollNo:110,name:"Chetan",course:"BCA",semester:3,marks:87,attendance:89},
  {rollNo:111,name:"Deepak",course:"BCA",semester:3,marks:66,attendance:72},
  {rollNo:112,name:"Diya",course:"BCA",semester:3,marks:92,attendance:94},
  {rollNo:113,name:"Divya",course:"BCA",semester:3,marks:81,attendance:86},
  {rollNo:114,name:"Gaurav",course:"BCA",semester:3,marks:74,attendance:78},
  {rollNo:115,name:"Ishita",course:"BCA",semester:3,marks:89,attendance:93},
  {rollNo:116,name:"Karan",course:"BCA",semester:3,marks:77,attendance:82},
  {rollNo:117,name:"Kavya",course:"BCA",semester:3,marks:93,attendance:96},
  {rollNo:118,name:"Khushi",course:"BCA",semester:3,marks:68,attendance:75},
  {rollNo:119,name:"Manish",course:"BCA",semester:3,marks:85,attendance:90},
  {rollNo:120,name:"Meera",course:"BCA",semester:3,marks:90,attendance:92},
  {rollNo:121,name:"Mohit",course:"BCA",semester:3,marks:71,attendance:80},
  {rollNo:122,name:"Muskan",course:"BCA",semester:3,marks:86,attendance:87},
  {rollNo:123,name:"Naman",course:"BCA",semester:3,marks:78,attendance:83},
  {rollNo:124,name:"Neha",course:"BCA",semester:3,marks:94,attendance:98},
  {rollNo:125,name:"Nikhil",course:"BCA",semester:3,marks:80,attendance:85},
  {rollNo:126,name:"Pallavi",course:"BCA",semester:3,marks:72,attendance:77},
  {rollNo:127,name:"Pankaj",course:"BCA",semester:3,marks:88,attendance:91},
  {rollNo:128,name:"Pooja",course:"BCA",semester:3,marks:83,attendance:88},
  {rollNo:129,name:"Pragya",course:"BCA",semester:3,marks:67,attendance:74},
  {rollNo:130,name:"Priya",course:"BCA",semester:3,marks:96,attendance:97},
  {rollNo:131,name:"Rahul",course:"BCA",semester:3,marks:75,attendance:81},
  {rollNo:132,name:"Riya",course:"BCA",semester:3,marks:89,attendance:93},
  {rollNo:133,name:"Rohit",course:"BCA",semester:3,marks:91,attendance:95},
  {rollNo:134,name:"Sakshi",course:"BCA",semester:3,marks:70,attendance:76},
  {rollNo:135,name:"Sameer",course:"BCA",semester:3,marks:84,attendance:89},
  {rollNo:136,name:"Shivam",course:"BCA",semester:3,marks:79,attendance:84},
  {rollNo:137,name:"Shreya",course:"BCA",semester:3,marks:87,attendance:90},
  {rollNo:138,name:"Simran",course:"BCA",semester:3,marks:92,attendance:96},
  {rollNo:139,name:"Sonal",course:"BCA",semester:3,marks:73,attendance:79},
  {rollNo:140,name:"Tanmay",course:"BCA",semester:3,marks:85,attendance:86},
])
```

Figure 2: Multiple student documents inserted using insertMany(), showing the student records with roll numbers, marks and attendance.

SCREENSHOT / EXPERIMENTAL EVIDENCE

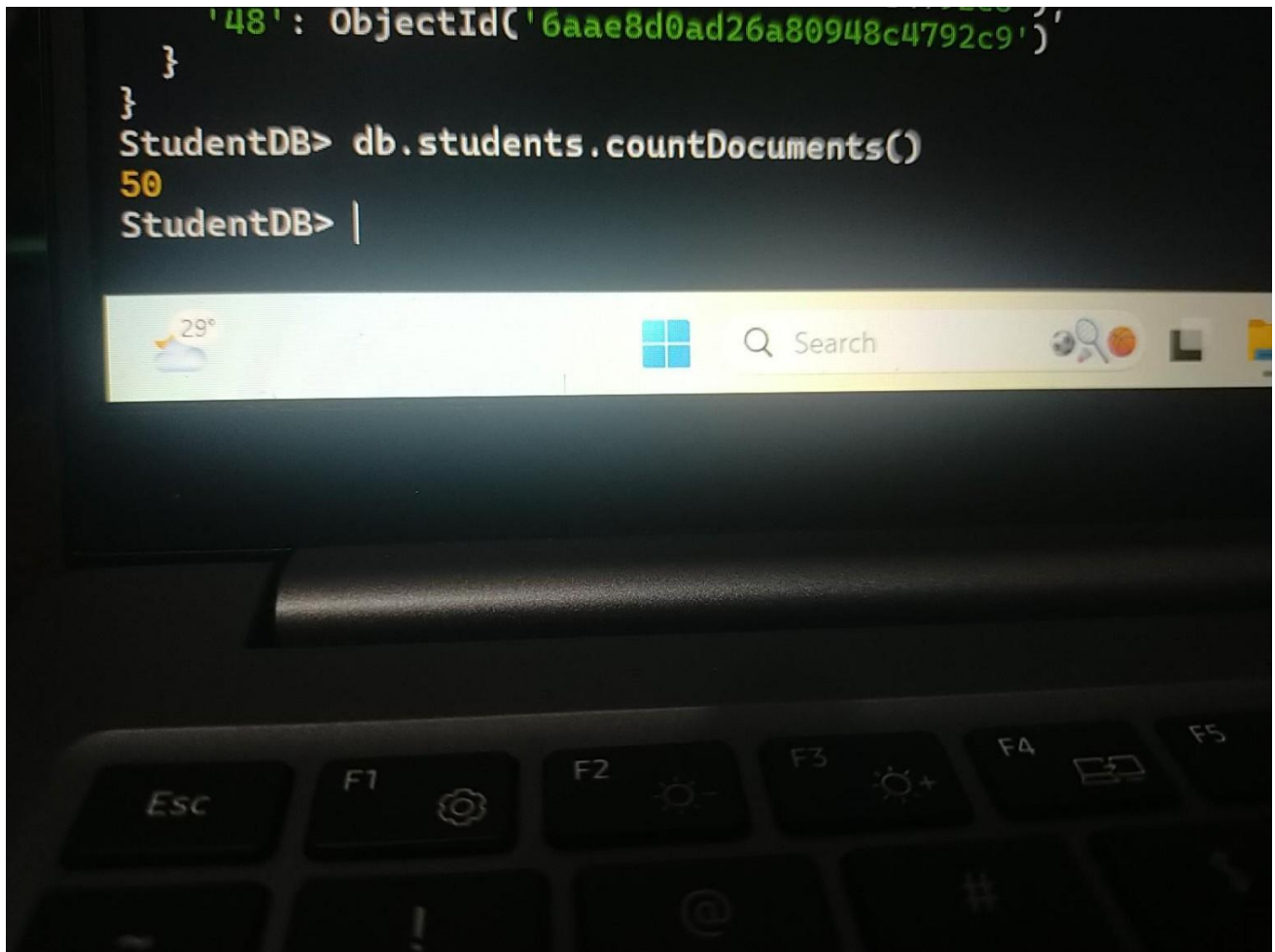


Figure 3: Collection verification using countDocuments(). The executed collection contains 50 student documents at this stage.

SCREENSHOT / EXPERIMENTAL EVIDENCE

```
[
  {
    _id: ObjectId('6aae8c72d26a80948c479298'),
    rollNo: 101,
    name: 'Aarav',
    course: 'BCA',
    semester: 3,
    marks: 82,
    attendance: 92
  },
  {
    _id: ObjectId('6aae8d0ad26a80948c479299'),
    rollNo: 102,
    name: 'Aditi',
    course: 'BCA',
    semester: 3,
    marks: 91,
    attendance: 95
  },
  {
    _id: ObjectId('6aae8d0ad26a80948c47929a'),
    rollNo: 103,
    name: 'Aditya',
    course: 'BCA',
    semester: 3,
    marks: 76,
    attendance: 81
  },
  {
    _id: ObjectId('6aae8d0ad26a80948c47929b'),
    rollNo: 104,
    name: 'Akash',
    course: 'BCA',
    semester: 3,
    marks: 88,
    attendance: 88
  }
]
```

Figure 4: Student documents displayed from the students collection using find().

SCREENSHOT / EXPERIMENTAL EVIDENCE

```
}
]
StudentDB> db.students.updateOne(
|   { rollNo: 101 },
|   { $set: { marks: 90 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
StudentDB> db.students.findOne({ rollNo: 101 })
{
  _id: ObjectId('6aae8c72d26a80948c479298'),
  rollNo: 101,
  name: 'Aarav',
  course: 'BCA',
  semester: 3,
  marks: 90,
  attendance: 92
}
StudentDB> |
```

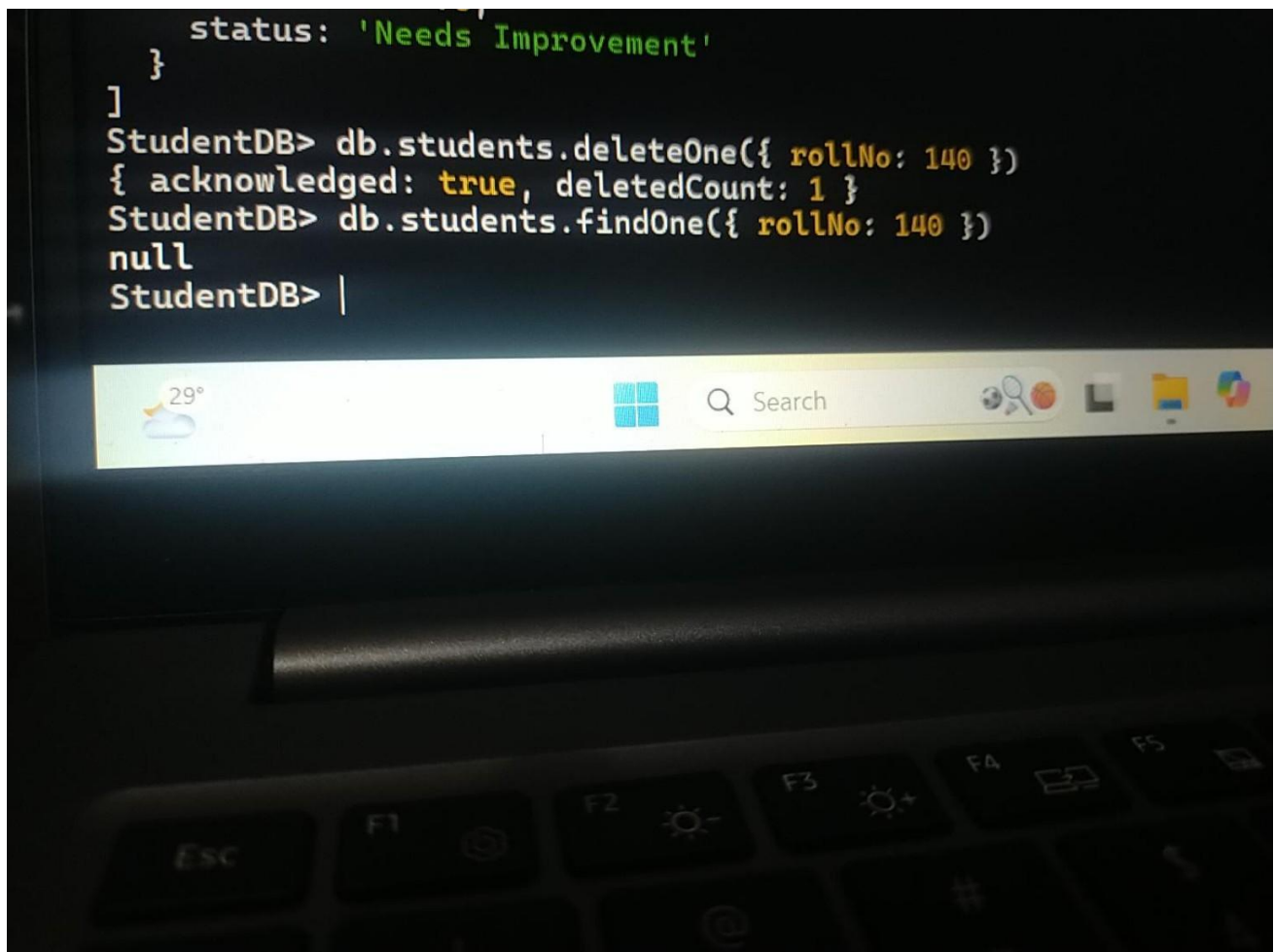
Figure 5: updateOne() successfully changed the marks of roll number 101 from 82 to 90, followed by findOne() verification.

SCREENSHOT / EXPERIMENTAL EVIDENCE

```
mongosh mongodb://127.0.0.1:27021/
> use StudentDB
StudentDB> db.students.updateMany(
  { attendance: { $lt: 80 } },
  { $set: { status: "Needs Improvement" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 11,
  modifiedCount: 11,
  upsertedCount: 0
}
StudentDB> db.students.find(
  { status: "Needs Improvement" }
)
[
  {
    _id: ObjectId('6aae8d0ad26a80948c47929c'),
    rollNo: 105,
    name: 'Ananya',
    course: 'BCA',
    semester: 3,
    marks: 69,
    attendance: 76,
    status: 'Needs Improvement'
  },
  {
    _id: ObjectId('6aae8d0ad26a80948c47929e'),
    rollNo: 107,
    name: 'Arjun',
    course: 'BCA',
    semester: 3,
    marks: 73,
    attendance: 79,
    status: 'Needs Improvement'
  },
  {
    _id: ObjectId('6aae8d0ad26a80948c4792a2'),
    rollNo: 111,
    name: 'Deepak',
    course: 'BCA',
    status: 'Needs Improvement'
  }
]
```

Figure 6: updateMany() successfully updated 11 records whose attendance was below 80 with status: 'Needs Improvement'.

SCREENSHOT / EXPERIMENTAL EVIDENCE



```
    status: 'Needs Improvement'
  }
]
StudentDB> db.students.deleteOne({ rollNo: 140 })
{ acknowledged: true, deletedCount: 1 }
StudentDB> db.students.findOne({ rollNo: 140 })
null
StudentDB> |
```

The screenshot shows a terminal window with a dark background. The text is displayed in a monospaced font. The first part shows a JSON document with a 'status' field. Then, a MongoDB command is executed to delete a document with 'rollNo: 140'. The response indicates that the document was successfully deleted. Finally, a verification command is run to find the document by 'rollNo: 140', which returns 'null'. The Windows taskbar is visible at the bottom of the terminal window, showing the weather (29°), search bar, and several application icons. Below the terminal window, a portion of a laptop keyboard is visible, showing keys like Esc, F1, F2, F3, F4, and F5.

Figure 7: deleteOne() successfully deleted roll number 140. Verification using findOne() returned null.

SCREENSHOT / EXPERIMENTAL EVIDENCE

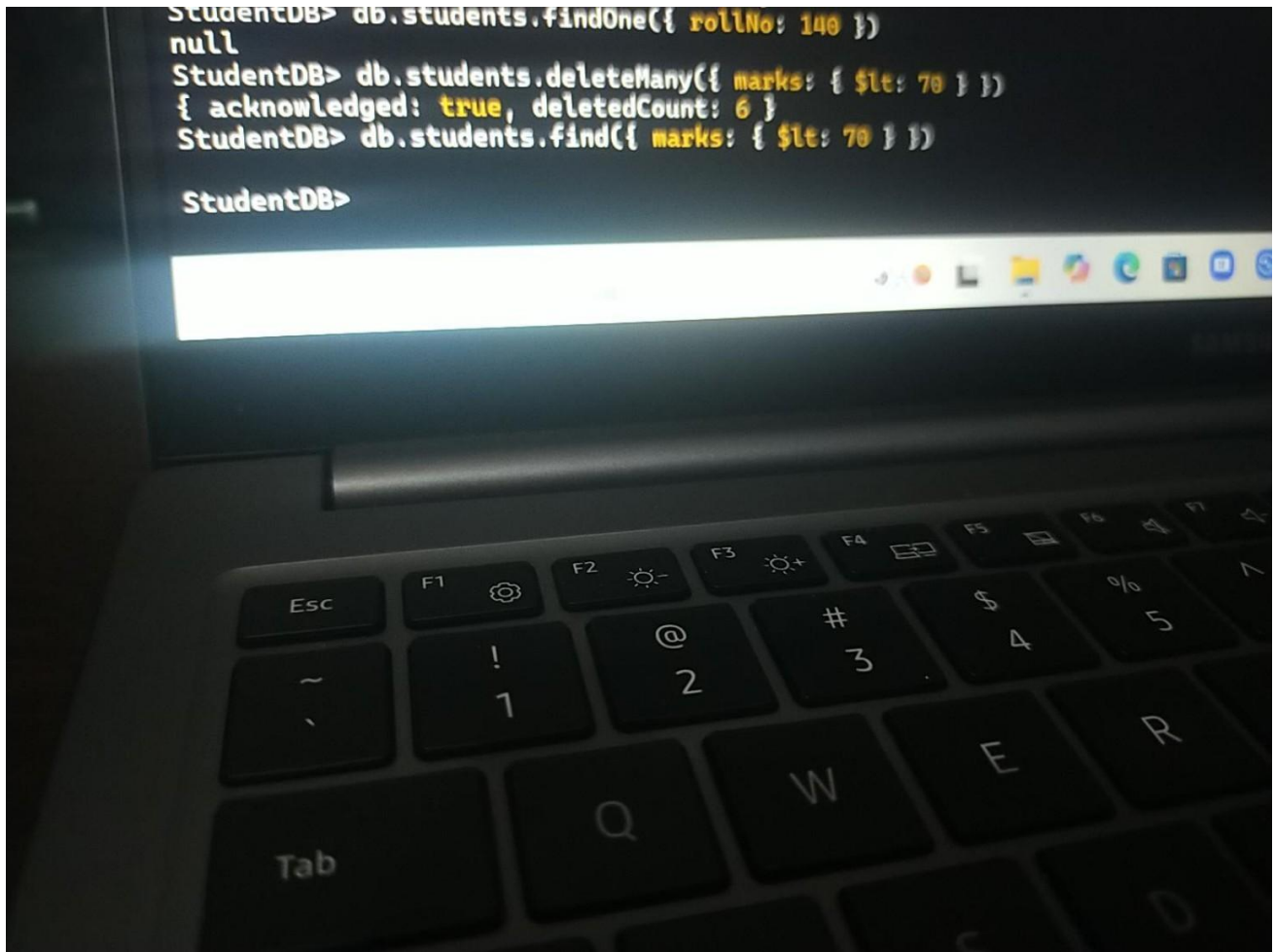


Figure 8: deleteMany() successfully removed 6 documents with marks below 70. The subsequent find() returned no matching records.

EXPECTED OUTPUT / OBSERVATION

Documents are inserted into the students collection, retrieved using `find()`, modified using `updateOne()/updateMany()`, and removed using `deleteOne()/deleteMany()`. The screenshots provide experimental evidence of the executed MongoDB commands and their outputs.

RESULT

Single and multiple student documents were inserted successfully. The documents were viewed, updated, and deleted successfully using MongoDB CRUD operations. The collection was also verified using `countDocuments()` and query results.

CONCLUSION

This practical demonstrated the basic CRUD operations required to manage MongoDB documents and collections effectively. It provided hands-on experience with document-based storage, querying, updating and deletion using MongoDB Shell.

THANK YOU !!

